

Homework – Subtract three values

This is a programming assignment that will be graded and scored as a homework since it is so simple.

Write a program for the LC3. This program should be named ***hwSub3Mem.hex***. This hex file will be a machine language program for the LC3 processor.

This program will be written as an ASCII hex file.

DON'T TRY TO ASSEMBLE IT WITH AN LC3 ASSEMBLER.

The hex file must contain only hex characters; 0-9 and A-F with no x, X, 0x, or 0X.

Each line should be a four-digit hex number. Do not include a 0x or x at the beginning of this number.

The first line should be the starting address in memory, use 3000 to be safe.

The last line should be a halt (F025)

The lines between the first and last line should do the following:

Using the value stored in the memory just below HALT as a starting point, subtract the values of the next three memory locations from that starting value. Store the result in the next memory location after the three.

Load the value from the location immediately after HALT into a register and then subtract the value stored the next three memory locations. Store the result in the fifth memory location after the halt.

... example instruction ...	
... example instruction ...	
HALT	The HALT is here.
10	Start with this value.
4	Subtract this value.
2	Subtract this value.
1	Subtract this value.
0	Store the result here.

As an example, IF YOUR HALT is at location 3006 (it may not be at 3006), your program should load the value from 3007 (10) into a register, then subtract the value stored in 3008 (4) from R0, then subtract the value stored in 3009 (2) from R0, then subtract the value stored in 300A (1) from R0, then store that value (3) in 300B.

ALL VALUES MUST BE STORED AS POSITIVE NUMBERS FOR TESTING. DO NOT STORE NEGATIVE NUMBERS.

Write your program in assembly. Convert it manually to hex. Put the hex in hwSub3Mem.hex.

Test your program using the simulator. Make sure to test with numbers OTHER than the ones we discussed in class.

To test your program, compile it and load it into the simulator. **You can change the values 10, 5, 2 in the hex file and the reset OR you can simply change them manually in the simulator.**

Your program MUST NOT MODIFY THE VALUES IN ANY MEMORY LOCATION OTHER THAN the one memory location specified in any way.

Run your program and note that it stores the correct value in the FIFTH memory location after your HALT wherever your HALT may be. Make sure your program reaches and executes the HALT instruction.

Submit your hwSub3Mem.hex file to Web-CAT. Make sure you did not name it hwSub3Mem.txt or that Notepad or WordPad did not rename it hwSub3Mem.hex.txt.

NOTE: You will lose 1 point for every submission over 5. You will lose 10 points for each day late. Submitting more than 2 days late will result in a zero.

Late days start at 9:00 am. So, submitting after 9:00 am on the due date will result in -10 points. Submitting after 9:00 am on the day after that will result in an additional -10 points. You will not be able to submit after 9:00 am on the day after that.

Here is an example test. YOU MUST DO MORE THAN TEST MORE THAN THE FOLLOWING. TEST MANY DIFFERENT NUMBERS. Put in zeros and negative numbers. It should work regardless.

1. Load your program into the simulator but do not run it yet.
2. Scroll in the simulator memory section down to memory location after the halt
3. Double click the memory location below HALT and store the number 10 in that memory location (10 not 0x10).
4. Double click on the next memory and change the value to 4.
5. Double click on the next memory and change the value to 2.
6. Double click on the next memory and change the value to 1.
7. Run your program.
8. It should reach the halt and you should see a message about halting the processor in the output area of the simulator.
9. You should see the number 3 stored in the memory location below where the 1 is stored because: $10 - 4 - 2 - 1 = 3$.
10. If you do not see the number 3 in that memory, then repeat the above and step through your program one step at a time and see where it went wrong. Verify at each instruction that the thing you expected to happen actually happened. The instruction that does something unexpected is most likely your bug. Once you have corrected the program, reload and start over. You may have more than one bug